Lab Work for Week 3: Django Views and Templates

Objective

By the end of this lab, students will:

1. Understand Django views and templates.
2. Create function-based views (FBVs) to handle requests.
3. Render dynamic content using Django templates.
4. Pass data from views to templates.

Step-by-Step Lab Tasks

---

1. Create Views for Course Listing and Details

1. **Task**: Create a view to list all courses.
    - Open views.py in the courses app and add the following code:

```
from django.shortcuts import render
from .models import Course

def course_list(request):
  courses = Course.objects.all()
  return render(request,'courses/course_list.html',{'courses': courses})
```

2. **Task**: Create a view to display course details, including its lessons.
    - Add the following to views.py:

```
from django.shortcuts import get_object_or_404

def course_detail(request, course_id):
  course = get_object_or_404(Course, id = course_id)
  lessons = course.lessons.all()
  return render(request,'courses
            /course_detail.html',{'course': course,'lessons': lessons})
```

---

2. Configure URLs for the Views

1. **Task**: Define URL patterns for the views.
    - Create a urls.py file in the courses app and add the following:

```
from django.urls import path
from . import views

urlpatterns = [
  path('', views.course_list, name='course_list'),
  path('<int:course_id>/', views.course_detail, name='course_detail'),
]
```

2. **Task**: Include the courses app URLs in the project's main urls.py.
    - Open EduLearn/urls.py and modify it:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
  path('admin/', admin.site.urls),
  path('', include('courses.urls')),
]
```

---

3. Create Templates for the Views

1. **Task**: Create a templates folder in the courses app directory.
    - Inside templates, create a folder named courses.
2. **Task**: Create a template for the course list.
    - Create course_list.html in courses/templates/courses:

```
<!DOCTYPE html>
<html>
<head>
  <title>Courses</title>
</head>
<body>
```

```
  < h1 > Available Courses </h1 >
  < ul >
    {% for course in courses %}
      < li >
        < a href = "{{ course.id }}/" > {{ course.title }} </a >
        < p > {{ course.description }} </p >
      </li >
    {% endfor %}
  </ul >
</body >
</html >
```

3. **Task**: Create a template for course details.
    - Create course_detail.html in courses/templates/courses:

```
< ! DOCTYPE html >
< html >
< head >
  < title > {{ course.title }} </title >
</head >
< body >
  < h1 > {{ course.title }} </h1 >
  < p > {{ course.description }} </p >
  < h2 > Lessons </h2 >
  < ul >
    {% for lesson in lessons %}
      < li > {{ lesson.title }} </li >
    {% endfor %}
  </ul >
  < a href = "/" > Back to Courses </a >
</body >
</html >
```

4. Test the Application

1. **Task**: Run the development server.

   python manage.py runserver

2. **Task**: Visit the following URLs in your browser:
   - http://127.0.0.1:8000/ - Displays the list of courses.
   - http://127.0.0.1:8000/<course_id>/ - Displays details and lessons for a specific course.
3. **Task**: Verify that:
   - All courses are listed with their details on the course list page.
   - Clicking a course title navigates to its detail page, displaying lessons.

Lab Deliverables

1. Code for views.py, urls.py, and the templates (course_list.html and course_detail.html).
2. Screenshots of:
   - The course list page showing at least two courses.
   - A course detail page showing its description and lessons.